

Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

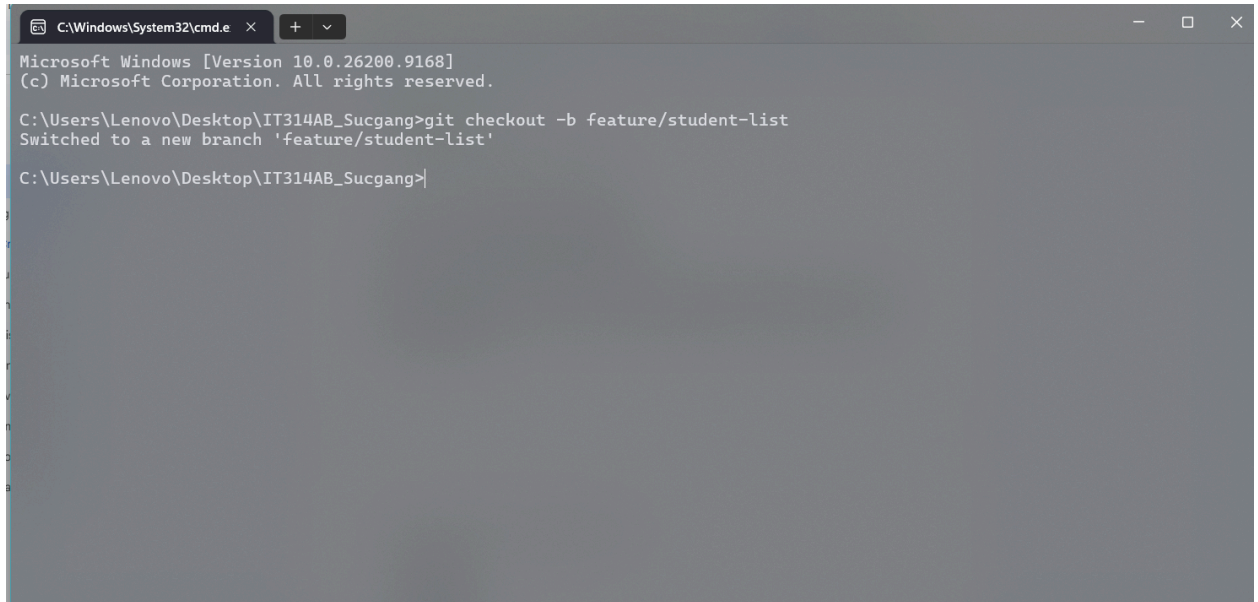
Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\Desktop\IT314AB_Sucgang>git checkout -b feature/student-list
Switched to a new branch 'feature/student-list'

C:\Users\Lenovo\Desktop\IT314AB_Sucgang>
```

🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart **List**.

Research what a Dart **List** is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list
- At least five student entries

```

1 class Student {
2   final String imagePath;
3   final String name;
4   final String course;
5   final String yearLevel;
6   final int age;
7   final String hobby;
8
9   const Student({
10    required this.imagePath,
11    required this.name,
12    required this.course,
13    required this.yearLevel,
14    required this.age,
15    required this.hobby,
16  });
17 }
18

```

```

1 import 'package:my_first_flutter_app/Models/student.dart';
2
3 const List<Student> students = [
4   Student(
5     imagePath: 'asset/images/Cat.jpg',
6     name: 'John the Cat',
7     course: 'BSIT',
8     yearLevel: '3rd Year',
9     age: 21,
10    hobby: 'Watching football',
11  ),
12   Student(
13     imagePath: 'asset/images/cat2.jpg',
14     name: 'Mittens',
15     course: 'BSIT',
16     yearLevel: '2nd Year',
17     age: 20,
18     hobby: 'Playing with yarn',
19   ),
20   Student(
21     imagePath: 'asset/images/cat3.jpg',
22     name: 'Whiskers',
23     course: 'BSCS',
24     yearLevel: '1st Year',
25     age: 19,
26     hobby: 'Sleeping in boxes',
27   ),
28   Student(
29     imagePath: 'asset/images/cat4.jpg',
30     name: 'Shadow',
31     course: 'BSIS',
32     yearLevel: '4th Year',
33     age: 22,
34     hobby: 'Chasing laser pointers',
35   ),
36   Student(
37     imagePath: 'asset/images/cat5.jpg',
38     name: 'Milo',
39     course: 'BSIT'
40   )

```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)



```
mainAxisSize: MainAxisSize.min,  
children: [  
  ProfileWidget(),  
  Profile2Widget(),  
  Profile3Widget(),  
  Profile4Widget(),  
  Profile5Widget(),  
  Profile6Widget(),  
  Profile7Widget(),  
  Profile8Widget(),  
],  
), // Column
```

🚩 Flag 3 — Discover **ListView**

Objective: Make your student directory scrollable using **ListView**.

Research Flutter's **ListView**.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

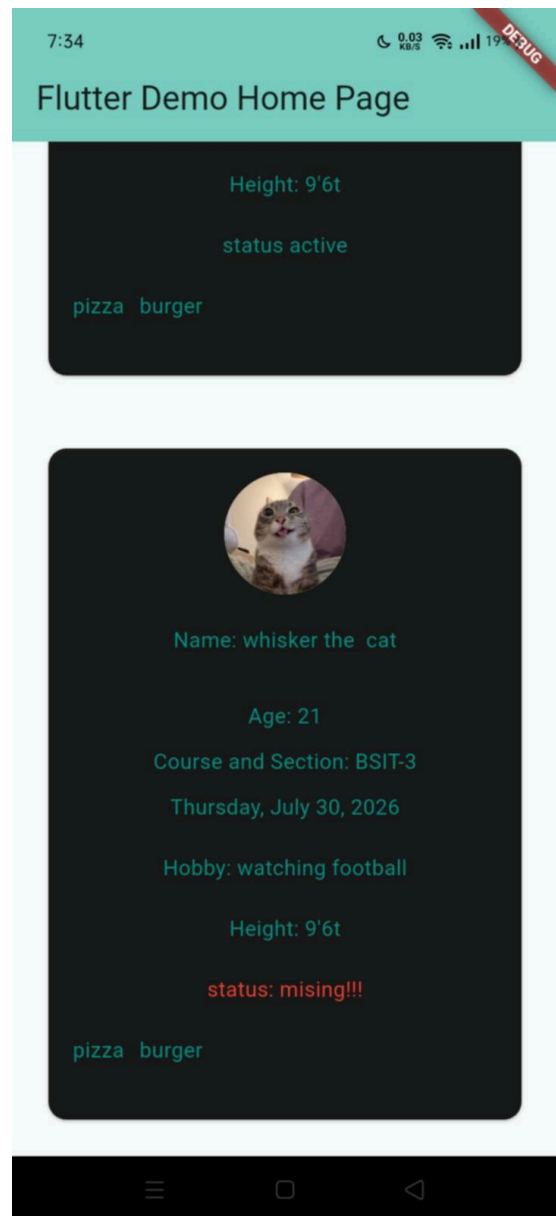
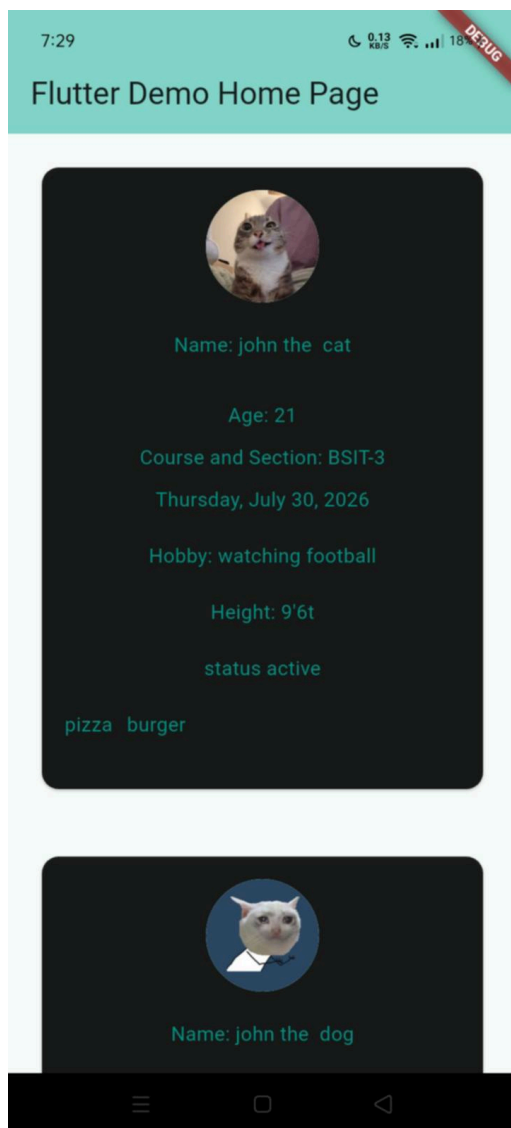
The user should be able to:

- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- `ListView` in your code
- Top of the application
- Bottom portion showing later students

```
body: Center(
  // Center is a layout widget.
  // in the middle of the parent
  child: ListView(
    children: [
      ProfileWidget(),
      Profile2Widget(),
```



🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

Screenshot:

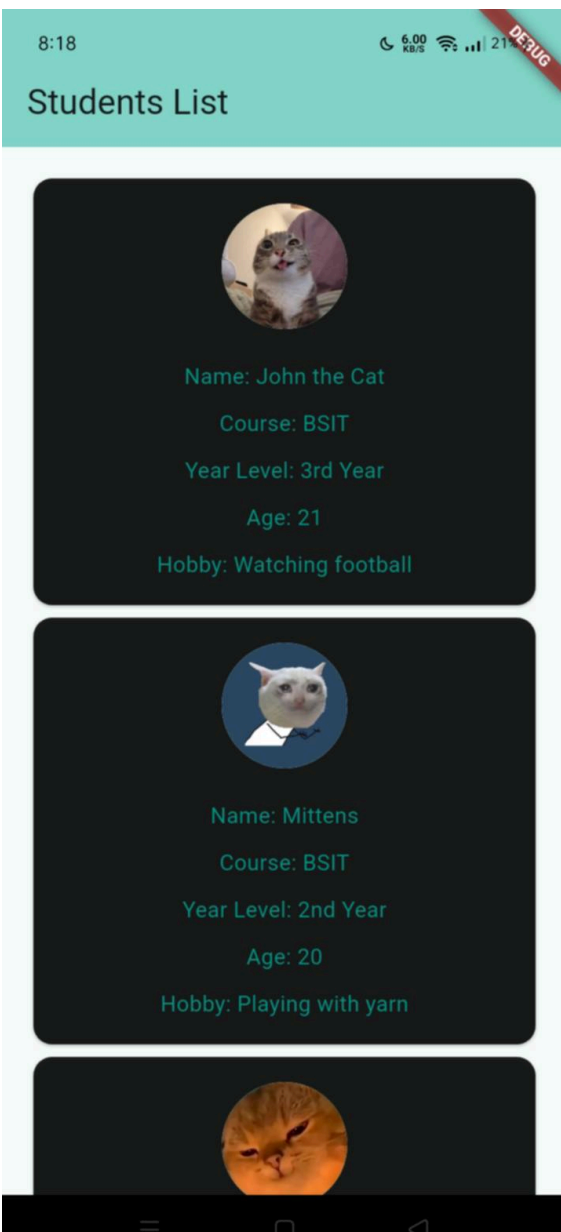
- `ListView.builder` in your code
- Student list
- Newly generated student appearing in the app

```
body: ListView.builder(  
  padding: const EdgeInsets.all(16),  
  itemCount: students.length,  
  itemBuilder: (context, index) {  
    final student = students[index];  
    return StudentCard(student: student);  
  },  
) // ListView.builder
```

```

my_first_flutter_app > lib > data > student_data.dart > students
1  import 'package:my_first_flutter_app/Models/student.dart';
2
3  const List<Student> students = [
4    Student(
5      imagePath: 'asset/images/Cat.jpg',
6      name: 'John the Cat',
7      course: 'BSIT',
8      yearLevel: '3rd Year',
9      age: 21,
10     hobby: 'Watching football',
11   ),
12   Student(
13     imagePath: 'asset/images/cat2.jpg',
14     name: 'Mittens',
15     course: 'BSIT',
16     yearLevel: '2nd Year',
17     age: 20,
18     hobby: 'Playing with yarn',
19   ),
20   Student(
21     imagePath: 'asset/images/cat3.jpg',
22     name: 'Whiskers',
23     course: 'BSCS',
24     yearLevel: '1st Year',
25     age: 19,
26     hobby: 'Sleeping in boxes',
27   )

```



🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

Examples:

- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data
- Student Card displaying the additional fields

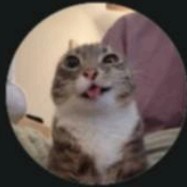
```
class Student {  
    final String imagePath;  
    final String name;  
    final String course;  
    final String yearLevel;  
    final int age;  
    final String hobby;  
    final String studentId;  
    final String email;  
    final String favoriteSubject;  
}
```


8:27

2.00 KB/S 22%

DEBUG

Students List



Name: John the Cat

Course: BSIT

Year Level: 3rd Year

Age: 21

Hobby: Watching football

Student ID: 2021-001

Email: john.the.cat@student.edu

Favorite Subject: Mathematics



Name: Mittens

Course: BSIT

Year Level: 2nd Year

Age: 20

Hobby: Playing with yarn

Student ID: 2021-002

🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

```
[]
```

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

- "No students found."
- "Student list is empty."

Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

- Empty student list
- Empty-state UI
- Application still functioning correctly

8:43

22.0 KB/s 24%

DEBUG

Students List



No students found.

Student list is empty.

Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a [List](#).

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

After sorting:

- Aaron
- John
- Maria
- Sophia

Your UI should immediately reflect the new order.

Screenshot:

- Student list before or after sorting
- Application displaying the sorted order

```
        email: 'new.student@student.edu',  
        favoriteSubject: 'Statistics',  
    ),  
];  
  
final List<Student> students = List.of(_studentsInOriginalOrder)  
    ..sort((a, b) => a.name.compareTo(b.name));
```

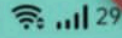
```

Student(
  imagePath: 'asset/images/Cat.jpg',
  name: 'John the Cat',
  course: 'BSIT',
  yearLevel: '3rd Year',
  age: 21,
  hobby: 'Watching football',
  studentId: '2021-001',
  email: 'john.the.cat@student.edu',
  favoriteSubject: 'Mathematics',
),
Student(
  imagePath: 'asset/images/cat2.jpg',
  name: 'Mittens',
  course: 'BSIT',
  yearLevel: '2nd Year',
  age: 20,
  hobby: 'Playing with yarn',
  studentId: '2021-002',
  email: 'mittens@student.edu',
  favoriteSubject: 'Physics',
),
Student(
  imagePath: 'asset/images/cat3.jpg',
  name: 'Whiskers',
  course: 'BSCS',
  yearLevel: '1st Year',
  age: 19,
  hobby: 'Sleeping in boxes',
  studentId: '2021-003',
  email: 'whiskers@student.edu',
  favoriteSubject: 'Computer Science',
),
Student(
  imagePath: 'asset/images/cat4.jpg',
  name: 'Shadow',
  course: 'BSIS',
  yearLevel: '4th Year',
  age: 22,
  hobby: 'Chasing laser pointers',
  studentId: '2021-004',
  email: 'shadow@student.edu',
  favoriteSubject: 'Information Systems',
),
Student(
  imagePath: 'asset/images/cat5.jpg',
  name: 'Milo',
  course: 'BSIT',
  yearLevel: '3rd Year',
  age: 21,
  hobby: 'Bird watching',
  studentId: '2021-005',
  email: 'milo@student.edu',
  favoriteSubject: 'Mathematics',
),
Student(
  imagePath: 'asset/images/cat2.jpg',
  name: 'New Student',
  course: 'BSIT',
  yearLevel: '3rd Year',
  age: 21,
  hobby: 'New hobby',
  studentId: '2021-006',
  email: 'new.student@student.edu',
  favoriteSubject: 'Statistics',
),
];

```

9:07

0.11 KB/s



29%
DEBUG

Students List

Favorite Subject: Mathematics



Name: Milo

Course: BSIT

Year Level: 3rd Year

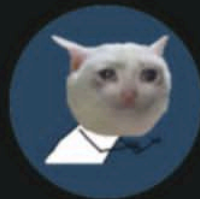
Age: 21

Hobby: Bird watching

Student ID: 2021-005

Email: milo@student.edu

Favorite Subject: Mathematics



Name: Mittens

Course: BSIT

Year Level: 2nd Year

Age: 20

Hobby: Playing with yarn

Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
C:\Users\Lenovo\Desktop\IT314AB_Sucgang>git push --set-upstream origin feature/student-list
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 87.44 KiB | 17.49 MiB/s, done.
Total 14 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'feature/student-list' on GitHub by visiting:
remote:   https://github.com/Ezik143/IT314AB_Sucgang/pull/new/feature/student-list
remote:
To https://github.com/Ezik143/IT314AB_Sucgang.git
 * [new branch]      feature/student-list -> feature/student-list
branch 'feature/student-list' set up to track 'origin/feature/student-list'.
C:\Users\Lenovo\Desktop\IT314AB_Sucgang>
```